

Architectural Structures and Views

Session 3

Architectural Structures and Views

- The **neurologist**, the, **orthopedist**, **cardiologists**, **Pediatrics** have different view of the structure of a human body.
- Although these views are differently pictures and have very different properties, all are inherently related: **Together they describe the architecture of the human body.**

Architectural Structures and Views

- It's the same approach with software. **Modern systems are complex to grasp them all at once.**
- At a particular moment, we concentrate on only one of the software structures.
- To communicate meaningfully about an architecture, we must make clear **which structure or structures we are discussing at the moment—which view we are taking of the architecture.**

Architectural Structures and Views

1. A **view** is a representation of a logical set of **architectural elements** and the **relations** among them.

(Not *all* architectural elements -- *some* of them.)

2. A **structure** is the set of **elements** itself, as they exist in software or hardware.
3. These terms are often used interchangeably.

Architectural Structures and Views

Example: a **module structure** is the set of the system's **modules** and their organization.

A **module view** is the representation of that structure, as documented by and used by some system stakeholders.

Analogy:

The building itself, classrooms, labs, library, offices → **Structure**

The floor plan shown to students → **View**

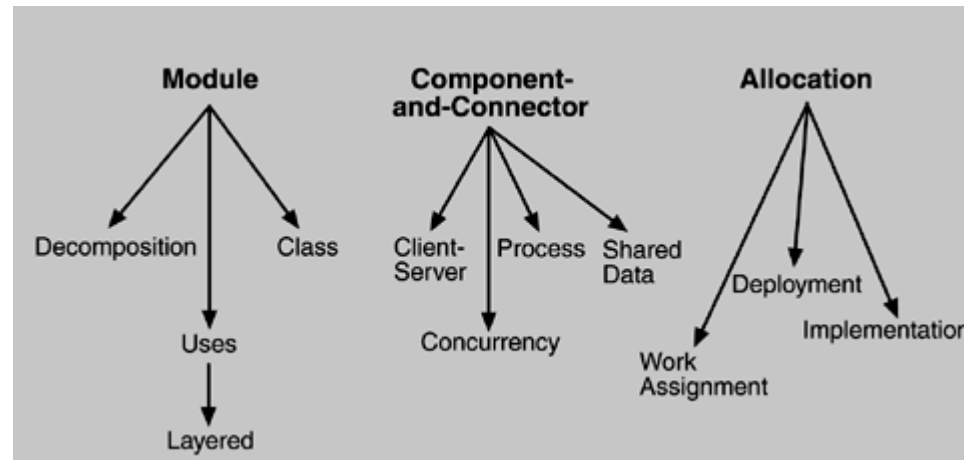
The building is the real thing.

The map is just a way of showing it.

Architectural Structures and Views

Architectural structures can be **divided into three groups**, depending on the kind of the elements they show:

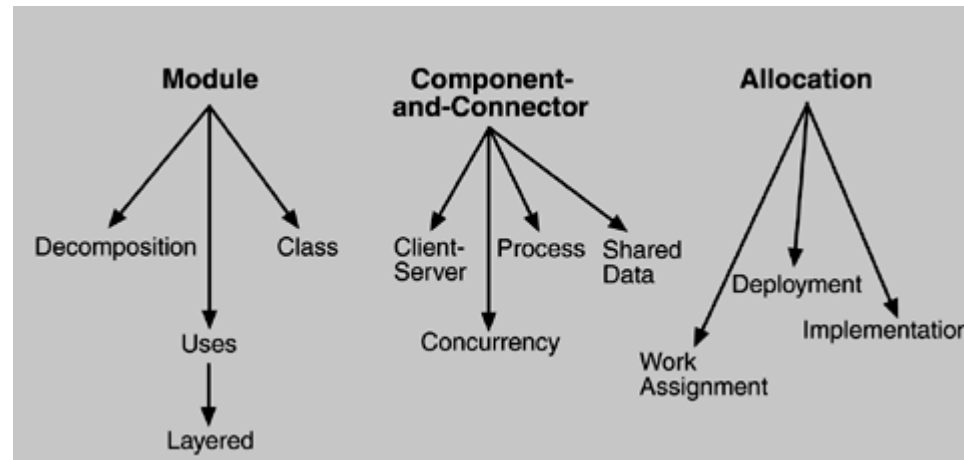
- Module structures - how the software is divided into parts.
- Component-and-connector structures- how components interact at runtime.
- Allocation structures - where software runs or who develops it.



Architectural Structures and Views

Architectural structures can be **divided into three groups**, depending on the kind of the elements they show:

- **Module structures - how the software is divided into parts.**
- Component-and-connector structures- how components interact at runtime.
- Allocation structures - where software runs or who develops it.



Module structures

- The **elements** in a module structure are the **modules**, which are units of implementation.
- Modules are decided based on the **functions module performs**.

Module structures allow us to answer questions?

- What is the primary functional responsibility assigned to each module?
- What other software elements in a module allowed to use?
- What other software does it actually use?
- What modules are related to other modules (by generalization or specialization) relationships?

Module structures

Module-based structures include the following:

- Decomposition
- Uses
- Layered
- Class, or generalization

Decomposition Structure

- Here the units are modules.
- They are related to each other by the "is a sub module of " relation.
- Larger modules are decomposed into smaller ones recursively until they are small enough to be easily understood.
- Modules can be used as a starting point for design.
- Modules have associated products (i.e., interface specifications, code, test plans, etc.).

Modular decomposition

- Two modular decomposition models covered
 - An **object model** where the system is **decomposed** into **interacting objects**;
 - A **pipeline** or **data-flow** model where the system is **decomposed** into **functional modules** which transform inputs to outputs.

A student management system may contain:

- Login Module
- Registration Module
- Payment Module
- Report Module

A student management system may contain:

- Student Object
- Lecturer Object
- Subject Object
- ...

Object models

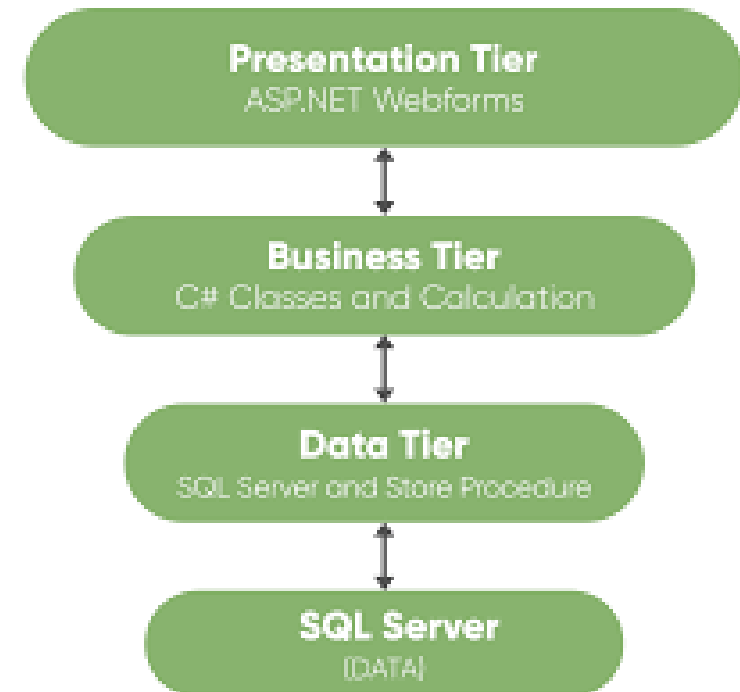
- Structure the system into a set **objects** with well-defined interfaces.
- Object-oriented decomposition is concerned with identifying **object classes**, their **attributes** and **operations**.
- When implemented, **objects** are created from these **classes** and **object operations** are using model **object behaviours**.

Uses Structure

- The units of this structure are modules, procedures or resources on the interfaces of modules.
- Units are **Procedures**
- The units are related by the **uses** relation
- The **uses structure** is used to engineer systems that can be **easily extended to add functionality**.

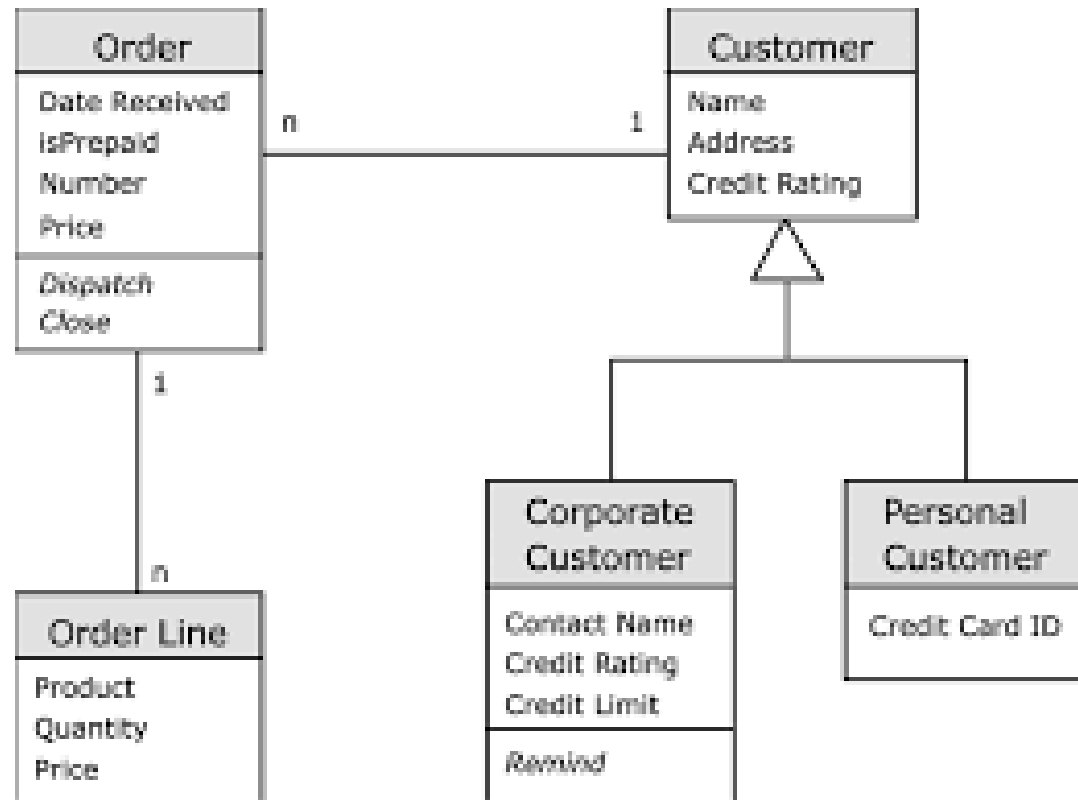
Layered Structure

- A layer is a logical set of **related functionality**.
- In a strictly layered structure, layer n may only use the services of layer $n - 1$.



Classes Structure

- The module units in this structure are called classes.
- The relation is "inherits-from" or "is-an-instance-of."



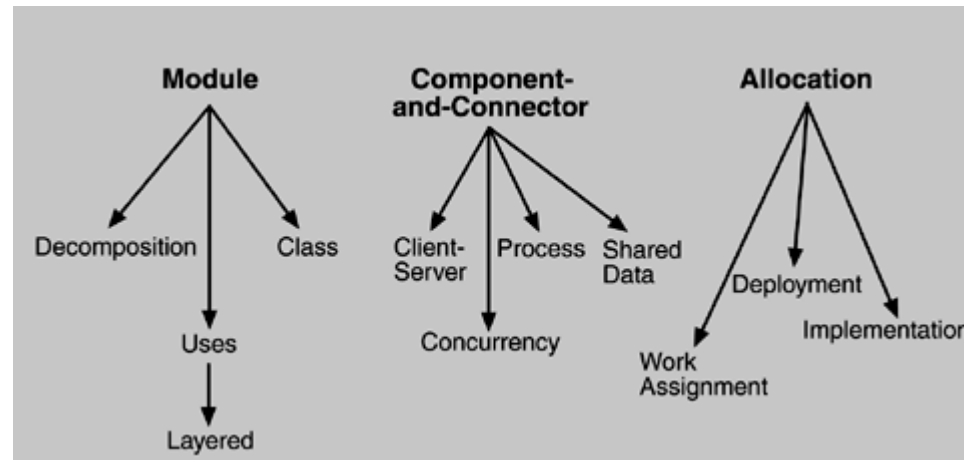
Module structures: Use Cases

- Understanding maintainability and flexibility.
- Supporting modular development.
- Facilitating team assignments.

Architectural Structures and Views

Architectural structures can be **divided into three groups**, depending on the kind of the elements they show:

- Module structures - how the software is divided into parts.
- **Component-and-connector structures- how components interact at runtime.**
- Allocation structures - where software runs or who develops it.



Component-and-connector structures

- The elements are **runtime components** and **connectors**
- The component and connector structure help us to answer questions such as:
 - ✓ What are the major executing components and how do they interact?
 - ✓ What are the major shared data stores?
 - ✓ Which parts of the system are replicated?
 - ✓ How does data progress through the system?
 - ✓ What parts of the system can run in parallel?

Component-and-connector structures

Component and connector structures include the following:

- Process
- concurrency
- Shared data
- Client server

Process Structure

- This structure is deals with the **dynamic** aspects of a running system
- The **units** here are **processes** or threads that are **connected** with each other by **communication**, **synchronization**, and/or, exclusion operators.
- The **relation** in this is **attachment**, showing how the **components** and **connectors** are hooked together.
- The process structure is important in helping to engineer a **system's execution performance and availability**

Concurrency Structure

- This component-and-connector structure allows the architect to determine **opportunities for parallelism** and the locations where **resource conflict** may occur.
- The connectors are "logical threads."
- A **logical thread** can be allocated to a **separate physical thread** later in the design process.

Shared data Structure

- This structure comprises components and connectors that **create, store,** and **access** persistent data.
- It shows how **data** is **produced** and **consumed** by **runtime software elements**, and it can be used to **ensure good performance and data integrity**.

Client server Structure

- If the system is built as a group of **cooperating clients and servers**, this is a good component-and-connector structure to clarify.
- The **components** are the clients and servers, and the **connectors** are **protocols** and **messages** they share to carry out the system's work.

Common Component-and-connector structures

- Client-Server View
 - Shows how clients and servers interact.
- Pipe-and-Filter View
 - Describes data flow between processing units.
- Event-Driven View
 - Focuses on event-based communication between components.
- Peer-to-Peer View
 - Highlights direct interactions among components.

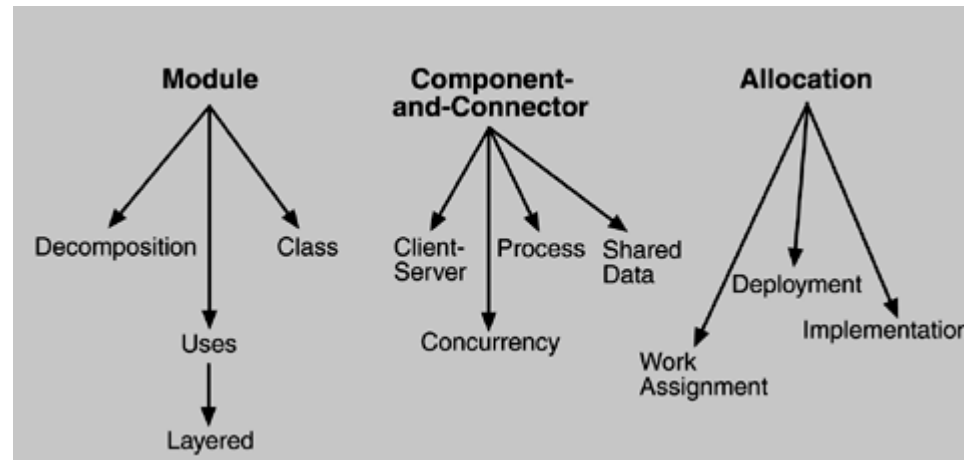
C&C Use Cases

- Understanding runtime behaviors.
- Analyzing performance and scalability.
- Identifying communication bottlenecks.

Architectural Structures and Views

Architectural structures can be **divided into three groups**, depending on the kind of the elements they show:

- Module structures - how the software is divided into parts.
- Component-and-connector structures- how components interact at runtime.
- **Allocation structures - where software runs or who develops it.**



Allocation structures

Allocation structures show the relationship between the **software elements and the elements in one or more external environments in which the software is created and executed.**

They answer questions such as:

- ✓ What processor does each software element execute on?
- ✓ In what files is each element stored during development, testing, and system building?
- ✓ What is the assignment of software elements to development teams?

Allocation structures

- Allocation structures include the following:
 - ✓ Deployment
 - ✓ Implementation
 - ✓ Work assignment

Deployment Structure

- The deployment structure shows **how software is assigned to hardware-processing** and communication elements.
- The elements are **software** (usually a process from a component-and-connector view), **hardware** entities (processors), **and communication pathways**.
- **Relations** are "**allocated-to,**" showing on which physical units the **software elements** reside, and "**migrates-to,**" if the **allocation is dynamic**.

Deployment Structure

- This view allows an engineer to reason about **performance, data integrity, availability, and security**.
- It is of particular interest in **distributed or parallel systems**.

Implementation Structure

- ✓ This structure shows how **software elements** (usually modules) are mapped to the **file structure(s) in the system's development, integration, or configuration control environments.**

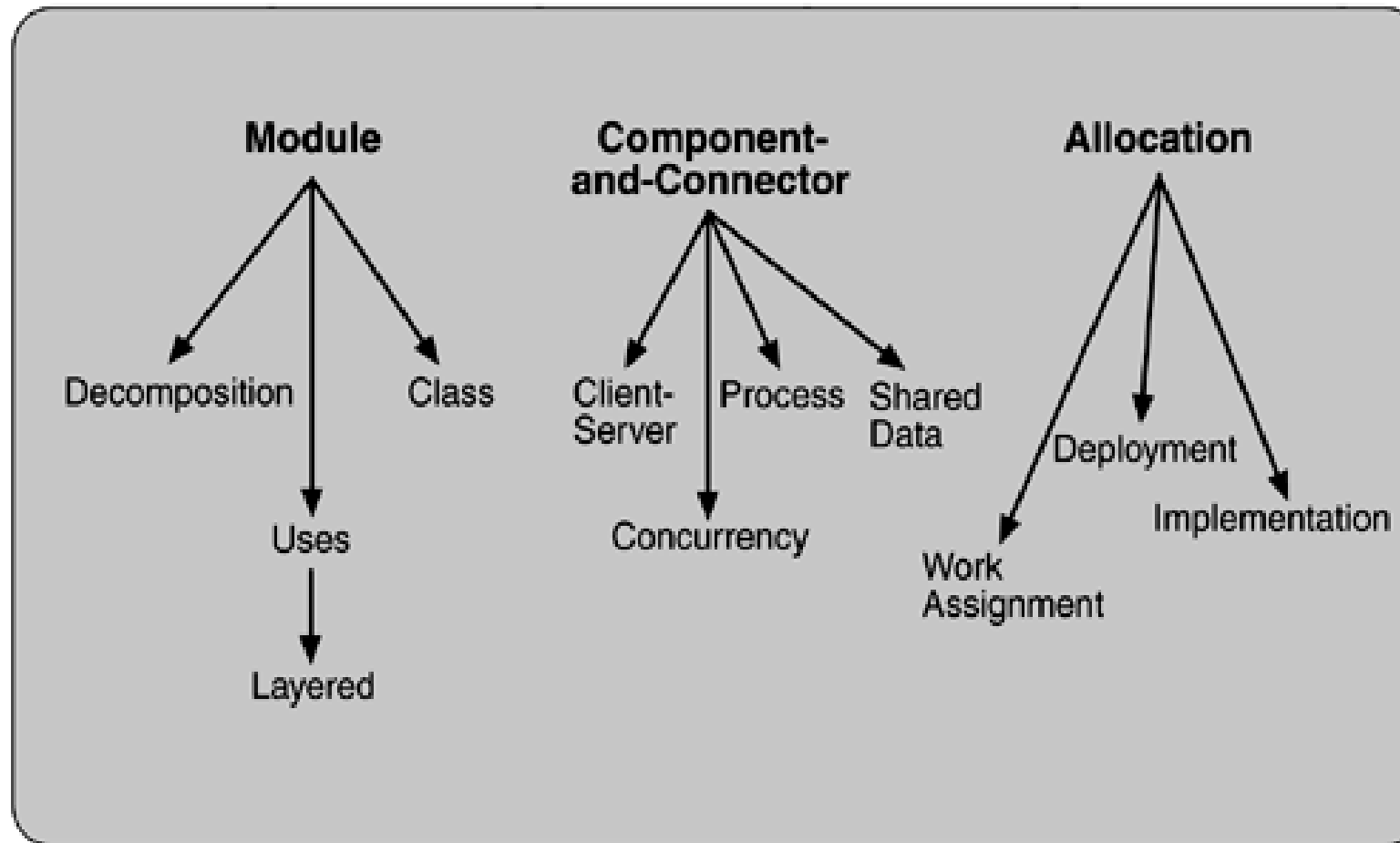
Work assignment Structure

- This structure assigns **responsibility** for implementing and **integrating** the **modules** to the appropriate **development teams**.
- The **architect** will know the **expertise** required on each team.

Allocation Structures: Use Cases

- Supporting deployment planning.
- Managing resources and hardware dependencies.
- Coordinating development efforts.

Common software architecture structures



A Simple Real-World Software Example

- Library Management System
 - Module structure → login module, payment module, report module
 - Client-server → browser + server + database
 - Deployment → app server on cloud, DB on another machine